Concurrent Programming:
Core Concepts

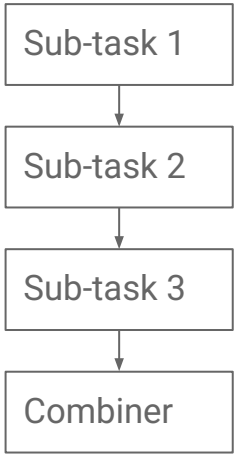
UNDERSTANDING > KNOWLEDGE

DANCING CLOUD SERVICES, LLC

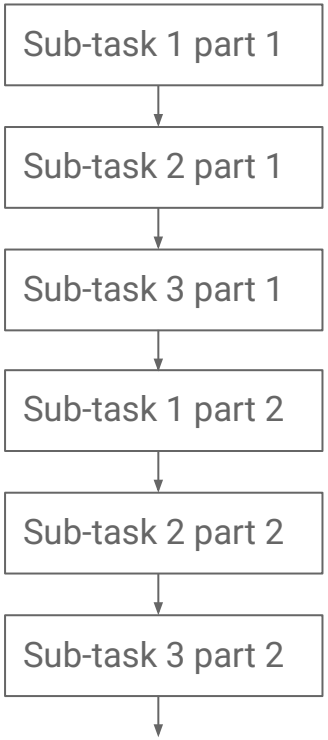
Why Threading?

- Better for describing sub-tasks that need to make progress concurrently
  - Provides separation of concerns in the process
- Allows use of parallel hardware
  - Probably the most natural approach for this

Sequential tasks



- Results only available after combiner finishes
- In this example, sub-task 1 doesn't need any data from any other sub-taskl



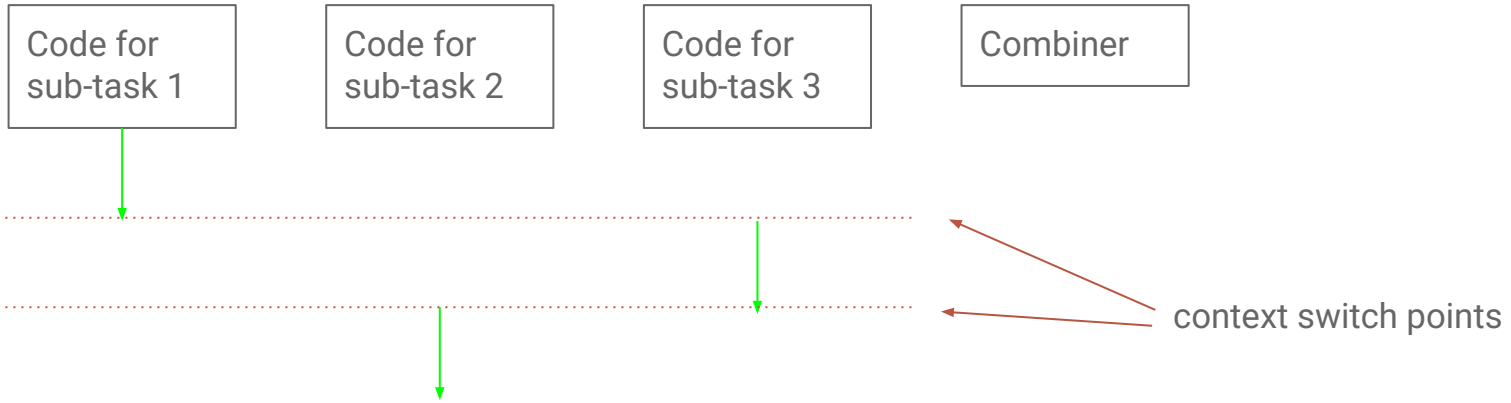
- Intermediate results might be supported
- Each task's description is split up, and mixed in with other tasks
  - Maintenance is horrific. Imagine simply trying to separate out one task

- Note that to cooperate on some larger task, sub-tasks must "share" data in some way
  - Easy when all in the same code

Concurrent tasks

- Define each task in isolation (mostly)
- Operating system (or perhaps execution engine, e.g. JVM) divides up the CPU time among the sub-tasks to make them look like they're all running "at once"
  - Commonly called the "scheduler"
- Programmer defines specific coordination points for passing data from one task to another (or more generally to share data)
  - These points (should) interact with the scheduler

Coordinating concurrent tasks



- Programmer defines specific coordination points for passing data from one task to another (or more generally to share data)
  - These points (should) interact with the scheduler
- Where the CPU stops work on one sub-task and picks up another, it's called a "context switch"

Context switching 1

When a context switch occurs, the system must store ***all*** the information that's specific to the current state of work so execution can pick up again later.

- Memory contents are considered stored anyway
  - If they're really in a cache, that cache will be flushed to memory before being overwritten, so it's considered "safe"
  - (In reality, this does have consequences, but the current model is making some assumptions)
- Contents of CPU registers must be preserved, as these will be quickly overwritten by the next task
  - Conventionally, this data is pushed onto the thread's stack

What's a thread's stack again?

- The "stack" is a Last-In First-Out (LIFO) structure
  - It provides a simple, but limited, method for memory allocation and release
  - It's ideally suited to local variables for subroutines (methods, functions, procedures–all equivalent in this context)
- Part of the area is "in use", and part is "free". The "stack pointer" indicates the start of the free region
- When a program makes a call, the stack pointer is moved to provide additional space for variables. When the call returns, the stack pointer is moved back, and execution jumps back to the point after the call
  - This all means that the stack also carries two additional elements of data: the "frame pointer" and the "program counter"
  - The frame pointer is "where the stack pointer was before this call", and the program counter is "the next instruction that should be executed after this call returns"

- NOTE: The following description can have variations, but is an adequate mental model for the needs of this course!

The stack in action 1

```go
func a() {
    var x = 10
    var y = "He"
    msg := b(x, y + "llo") ⇐ we are here
    fmt.Println(msg)
}

func b(p int, q string) string {
    return fmt.Sprintf(
        "%s repeated %d times\n", q, p)
}
```

Local variables {

Stack Pointer ⇒

x	10
y	"He"
free memory	

functioncall/functioncall.go

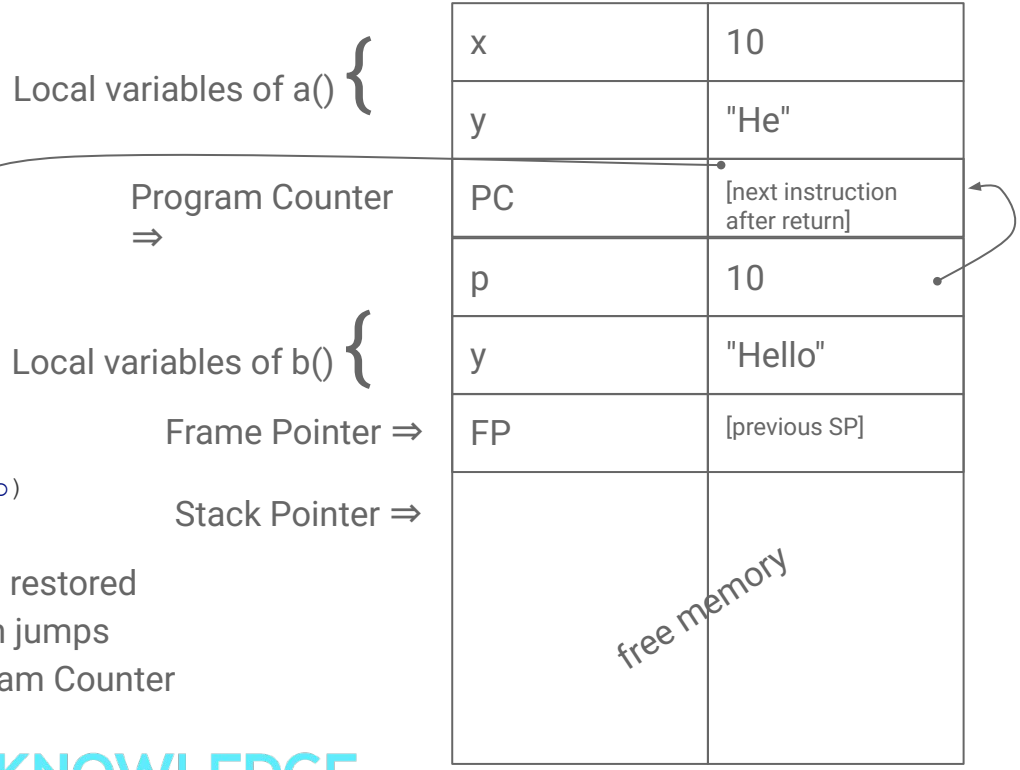
UNDERSTANDING > KNOWLEDGE
DANCING CLOUD SERVICES, LLC

The stack in action 2

```go
func a() {
    var x = 10
    var y = "He"
    msg := b(x, y + "llo")
    fmt.Println(msg)
}

func b(p int, q string) string {
                    ⇐ we are here
    return fmt.Sprintf(
        "%s repeated %d times\n", q, p)
}
```

- On function return the stack pointer is restored from the Frame Pointer, and execution jumps to the location described by the Program Counter stored in the stack



UNDERSTANDING > KNOWLEDGE
DANCING CLOUD SERVICES, LLC

Notes on the stack

- Details may vary, the key concepts are mostly consistent
- The arguments of a method are local variables, these and any explicit local variables are stored in the stack
- The Program Counter might describe the last instruction before the call, or the next after return
- The Program Counter might be at the "top" or "bottom" of the new stack frame
- Sometimes, values are actual values, sometimes they're pointers/references to where to find actual values
  - This depends on the language
- The returned value might be stored in the stack (e.g. in the caller's stack area as another local variable) or be handled using a CPU register – the model shown ignores this completely

- During a context switch, a new frame is (typically) made that will store CPU registers, along with a Program Counter to return to, and a Frame Pointer that will restore the stack's overall state to exactly what it was before the context switch

Context switching 2

- While the stack is used to store all the registers etc. there must still be a known storage location for the stack pointer from which the thread's state can be restored
- When execution switches to a different sub-task, some optimization information becomes stale:
  - instruction lookahead pipeline and any pre-executed instructions
  - CPU cache

- Overall, the detrimental effect on performance of a context switch can be dramatic, leading to an architectural guideline:

Minimize context switching

- This can be easier said than done, but if we start our designs with this in mind, it's easier than trying to fix the issue later!

Scalability of threads

- Threads often scale poorly, but this is not intrinsic to the *concept*
- Operating system threads are generalized:
  - Their stacks are usually large to support complex tasks with deep function calls
    - Mainstream O/Ss do not reallocate (grow) thread stacks, they simply kill that thread
    - The stack might benefit from virtual memory
  - Their stacks must be contiguous because they are used directly by the CPU
  - They have no knowledge of the task they're executing, so must preserve ***all*** the state of the CPU whether or not it's needed
  - Programmers rarely calculate the actual thread size needed, instead accepting the default (perhaps a few megabytes)
- By contrast if a threading system is built into an execution engine like the Java Virtual Machine or Common Language Runtime, it can be lighter weight
  - These systems have many names, including"virtual threads" (as in Java), "lightweight processes", "green threads" and probably more
  - The execution engine could have some knowledge of the code that is executing reducing the "context"
  - Stacks might be allocated in small, linked, chunks supporting vast numbers of tiny tasks

Amdahl's law

- Amdahl describes the effective throughput of a parallel system in which tasks intermittently cannot proceed (e.g. because they need data from some other task)
- The limitation on maximum throughput can be dramatic
- It can be helpful to think of the proportion of the task that has to be run sequentially; if this is 100% then only 1 CPU can be used. If it can all be run in parallel (which never happens in reality) an infinite number of CPUs can be put to work. In practice, some number of CPUs that runs all the runnable jobs at any given moment. More CPUs than that cannot contribute to the throughput of the overall task.
- The architectural guidance is to minimize thread interactions, so as to avoid making one thread wait for another
- In practice, this is similar to the guidance to minimizing context switching

Concurrent vs parallel execution

- In the model shown so far, we've not considered how many CPU cores might be available nor how to use them
- If the storage of context is perfect, and the CPUs have identical architecture (e.g. same instruction and register sets) then any CPU should be able to pick up where any other left off
  - And if the context isn't stored perfectly, this all fails anyway!
  - In this case, multiple CPUs can work on multiple threads and when one is idle, simply pick up any work that's ready to run.
  - This is called parallel execution

- Parallel execution is concurrent, but concurrent execution isn't parallel if only one CPU is in use
- There's more to say about concurrent-with-a-single-CPU versus parallel execution, but we'll get to that later

Using parallel hardware with a single task

- Unless your code is written in a way that describes separate sub-tasks, and the communication between them it typically cannot benefit from multiple CPU cores to speed up its execution.
- A human must usually describe how to split a big sequential task into independent subtasks, this is not usually automatable
  - Although this is what pre-execution, speculative execution, and runahead execution, might do at the CPU level
  - Move some parts of the sequential task alongside other parts shortening the overall case
- Some tasks (e.g. webserver) might be executed once per request for many concurrent requests
- It's not uncommon to be able to "unroll" a loop (e.g. some array operations such as image processing)

- Question: should you use parallel hardware to speed up the task you're working on?

3 problems of concurrent execution

If threads execute entirely independent tasks, parallelism is trivially easy for the programmer* (the O/S handles it). If threads must cooperate on a larger task, they must communicate data with one another and then the problems start. There are three overarching problems that affect correctness (two are related)

- Timing
  - If one thread is preparing data for another, the other thread must not read the memory before the write
  - If multiple data items will be transferred across the same memory location, the writer must not overwrite a previous data item until the reader has finished using it or definitely made a local copy
- Transactional integrity
  - If a thread is updating data or preparing compound data, other threads must take great care not to read inconsistent data in the middle of the updates
  - A timing control solution can be used to address this, but other more specific solutions are typically also provided
- Visibility
  - Surprisingly, writing to memory from one thread does not guarantee that another thread will read the written data

* If correctness depends on execution timing (i.e. a "real-time" system) this is not reliable. We won't look at this.

Timing issues

Timing problems are essentially:

- Don't read data that hasn't been written yet
- Don't overwrite data that is still needed

Libraries will provide tools that allow the effect of setting a "OK, ready for you" flag

- It's up to the programmer how to use these
- Typically, such tools can cause one thread to suspend execution (not use CPU power at all) until the other thread uses the too to indicate that the condition waited for has occurred.
- The library tool, and the underlying threading mechanism will not know about the condition, it only passes on the "ready now" message, so it's important that the programmer use these tools carefully and consistently

- Reading or writing without proper timing is called a ***race condition*** – note this term is often used in a slightly different, but overlapping context we'll address shortly

Transactional issues

Read-modify-write cycles

- Calculations are performed in the CPU, not in memory, therefore incrementing a value requires three sequential steps
  - Read data into the CPU (in a "register")
  - Increment the register
  - Write the new value to memory
- If another thread interacts with the memory in the middle of this process, bad results can occur

Compound updates

- If structured data is to be updated, each element of the structure must be written to memory in a separate operation
- This means that another thread could potentially see the data when one element is changed, but the rest has not changed
- e.g. while incrementing February 28, 2025 another thread might read February 1, or March 28

Visibility issues – 1

- When a thread updates a variable, we think of it as writing to memory at that moment, but optimizations make this model inaccurate. The effects of these optimizations are invisible in a single thread

- Within the same sequence of instructions, if the value isn't needed yet, the write might be delayed (or perhaps elided completely, leaving the value in a register for its next use)
  - This is a form of compiler optimization known as "instruction reordering" and in one sequence of instructions, we would never know this had happened.

reorder.Example1

Thread 1

```
y = 2 + 2; // some calculation
x = 1; // indicate value ready
```

Thread 2

```
while (x == 0)
 ; // wait for ready signal
System.out.println("value seen is " + y);
```

- In this example, the compilation of the code for Thread 1 can legitimately assign x before the calculation of y, or not assign x at all – but either of these changes wreck the expectations of Thread 2

Visibility issues – 2

- A modern CPU does not usually write data to memory, rather it writes to a cache
  - This is significantly faster, and subsequent reads are made from that cache and are faster too
  - Main memory might be updated later
- Critically, cache memory is usually "per-core" and is not shared among all cores
  - Updating other caches, writing to memory, or abandoning cache contents and re-reading from memory is expensive
- In practice the great majority of reads and writes of memory are only of interest to a single thread, and by extension to a single core
  - Therefore, caches are usually ***allowed to be inconsistent***, until a deliberate action in the code indicates that a particular data operation must be shared globally
  - Different CPUs have different cache models, and different instructions, which work in different ways. By contrast, a high level language that wishes to be platform independent must present a consistent approach on any CPU and OS

- Typically, programming languages provide a high level mechanism that addresses cache and instruction reordering aspects of visibility in a single utility

Race conditions again

- It's common that language constructs that address timing synchronization also address visibility issues
- The term ***race condition***, so far used to describe a failure to impose reliable timing between threads, is sometimes used to refer to a failure to guarantee proper visibility between threads
  - This is perhaps because the two can easily go hand in hand

Transactions

- Mechanisms for addressing transactional integrity almost always address timing and visibility too
  - A long-standing approach is to use a "locking" mechanism that enforces a "one at a time" access to either data, or code.
  - One, excellent, approach is to prepare data "in secret" and only make it available when fully stabilized
- Lock data, or lock code?
  - Locking data is less error prone, but is less commonly provided in the core of a language
  - Locking data is is more often available in core libraries, however, this likely only provides for the data structures provided by those libraries
  - For arbitrary, programmer-provided data, it's common to use a code-locking approach

Code-locking

- In this model, the programmer marks a region / a group of regions of code that are "critical sections"
  - These are "critical" with respect to the others in the group, not all areas marked as such
- The locking mechanism ensures that at most one thread is executing such a critical at any one time
- When a second thread attempts to enter a critical section, its execution is blocked until the first has exited
- ***Important note***, most such systems ***do not*** guarantee "fairness", that is, if thread a is in a critical section, thread b arrives at the start of a critical section and is blocked, then a third, thread c arrives and is blocked, it is possible that when thread a exits its critical section, ***thread c might run before thread b***
  - This is not just a theoretical possibility; designs that assume fair ordering will fail seemingly inexplicably!

- In this model, it's relatively easy for a programmer to overlook locking a section of code that needs it. Such an oversight will cause non-deterministic failures
- It's also possible to add unnecessary critical sections. This will reduce the throughput and performance of the system, and might cause deadlocks

Read-modify-write example – Java

transactions.ReadModifyWrite

```
class Incrementer implements Runnable {
  private long count = 0;
  @Override
  public void run() {
    for (int i = 0; i < 100_000_000; i++) {
      count++;
    }
    System.out.println("an incrementer has completed");
  }
  public long getCount() { return count; }
}
```

Read-modify-write example – Java

transactions.ReadModifyWrite

```
public class ReadModifyWrite {
  public static void main(String[] args) throws Throwable {
    Incrementer task = new Incrementer();
    Thread t1 = Thread.ofPlatform().start(task);
    Thread t2 = Thread.ofPlatform().start(task);
    t1.join();
    t2.join();
    System.out.println("Count value is: " + task.getCount());
  }
}
```

Read-modify-write example – Go

transaction/transaction.go

```go
var counter int64
var wait_group sync.WaitGroup
var l sync.Mutex

func incrementer() {
  defer wait_group.Done()
  for x := 0; x < 100_000_000; x++ {
    l.Lock()
    counter++
    l.Unlock()
  }
}
```

```go
func main() {
  wait_group.Add(2)
  go incrementer()
  go incrementer()
  wait_group.Wait()
  fmt.Println("value of counter is", counter)
}
```

Note that this style of locking, while valid, is not "idiomatic Go"

Mutate in private, publish when stable

- The most common variant of this idea is known variously as "producer-consumer", "pipeline architecture", and more recently as the "actor model"
- One thread is responsible for preparing data, and another for processing the next step
  - This is similar to a car production line, allowing multiple CPUs to be productive on what's otherwise a single, but repetitive, task
- A robust mechanism for transferring data between the producer and consumer threads is provided by a well-tested library
  - Often this is a thread-safe queue–we will refer to it as a queue, but other mechanisms might be encountered
- The producer creates and initializes data "in secret"; it's the only thread that has access to the data at first, so no threading issues exist.

Mutate in private, publish when stable

- When fully prepared, the data is transferred to the queue. After this:
  - the producer thread should not have access to any shared part of the data (this isn't always enforced)
  - at most one consumer thread can take an individual data item from the queue,
  - if multiple consumer threads are taking from the queue, no data item will be taken more than once
- Items put into the queue by any particular producer thread will be available for taking in the order they were added, but if multiple consumer threads are running, although a later item will be taken "in order" the consumer of a such a later item might complete processing it before the consumer of an earlier item completes processing that earlier item
- If multiple producers are putting into the queue, they will not conflict, but no particular ordering is guaranteed between the items of different producer threads

Queue requirements for reliable operation

For the producer-consumer model to work reliably, the queue must provide certain guarantees. The queue will normally be a library, or language, feature, so it has been well tested and can be assumed to be reliable

These behaviors of the queue address the timing problem:

- If a producer attempts to write to a full queue, either the queue grows, or the producer is blocked until space is available
  - The latter is likely preferable causes a kind of "auto-balancing" of workload, rather than an out-of-memory crash
- If a consumer attempts to read from an empty queue, the consumer is blocked until an item is available

This behavior of the queue addresses the visibility problem:

- There is a visibility relationship between the writing of an item into the queue and the taking of that item from the queue

This behavior of the design approach addresses the transactional problem

- The producer should not write data to the queue until it's fully stable, and should not read or modify it after writing

Variations on the producer-consumer approach

A number of variations of this approach exist, each having particular benefits

- If the language or programming model enforces immutable data, this removes the risk of shared data being mutated
- If the programming model copies data as it moves either into, or out of, the queue, and the data does not contain pointers (or references), then the data can be mutated freely by producer and consumer after being written into the queue, since these are now independent structures. Of course, each thread will not see changes by the other thread
- If the model copies data across a network, then this provides a simple model for distributed programming, not merely multi-threaded. If the local model copies data as above, then the change is transparent to the software

- These three elements together make a reasonable "elevator pitch" description of the "Actor" concept

Producer consumer example – Java – 1

prodcons.Example

```
Runnable producer = () -> {
 System.out.println("Producer starting");
 try {
   for (int i = 0; i < DATA_COUNT; i++) {
     int [] data = { 0, i };
     if (i < 1000) { Thread.sleep(1); }
     data[0] = i;
     queue.put(data);
   }
 } catch (InterruptedException ie) {
   System.out.println("Unexpected!!");
 }
 System.out.println("Producer completed");
};
```

```
BlockingQueue<int[]> queue =
  new ArrayBlockingQueue<>(10);
final int DATA_COUNT = 10_000;
```

UNDERSTANDING > KNOWLEDGE
DANCING CLOUD SERVICES, LLC

Producer consumer example – Java – 2

```
Runnable consumer = () -> {
 System.out.println("Consumer starting");
 try {
   for (int i = 0; i < DATA_COUNT; i++) {
     int [] data = queue.take();
     if (data[0] != data[1] || data[0] != i) {
       System.out.printf("**** ERROR\n");
     }
     if (i > DATA_COUNT - 1000) { Thread.sleep(1); }
   }
 } catch (InterruptedException io) {
   System.out.println("Unexpected!");
 }
 System.out.println("Consumer completed");
};
```

Producer-consumer example – Go – 1

```go
func producer(data_channel chan<- [2]int, control_channel chan<- bool) {
 count := rand.IntN(9_000) + 2000
 fmt.Printf("producer starting, will produce %d values\n", count)
 for c := 0; c < int(count); c++ {
   data := [2]int{0, c}
   if c < 1000 {
     time.Sleep(1 * time.Millisecond)
   }
   data[0] = c
   data_channel <- data
 }
 fmt.Println("producer completed, informing controller")
 control_channel <- true
}
```

Producer-consumer example – Go – 2

```go
func consumer(data_channel <-chan [2]int, control_channel chan bool) {
 fmt.Println("Consumer starting")
 c := 0
outer: for ; ; c++ {
   select {
   case data := <-data_channel:
     if data[0] != data[1] || data[0] != c { fmt.Printf("**** Error\n") }
     if c > 8500 { time.Sleep(1 * time.Millisecond) }
   case <-control_channel: break outer
   }
 }
 fmt.Printf("Consumer completed after processing %d items\n", c)
 control_channel <- true
}
```

Producer-consumer example – Go – 3

```go
func main() {
 data_channel := make(chan [2]int)
 producer_finished_channel := make(chan bool)
 shutdown_consumer_channel := make(chan bool)
 go producer(data_channel, producer_finished_channel)
 go consumer(data_channel, shutdown_consumer_channel)
 <-producer_finished_channel
 fmt.Println("Producer finished, informing consumer")
 shutdown_consumer_channel <- true
 fmt.Println("Consumer informed, awaiting acknowledgement of completion")
 <-shutdown_consumer_channel
 fmt.Println("main exiting")
}
```

The typical visibility model – 1

- A common description of visibility defines a kind of transitive visibility relationship between "actions"
  - We'll call this "is visible to"
- Actions include writing to memory, reading from memory, and a number of other operations
- A typical "core" of this model may be defined like this:
  - If action b immediately follows action a in normal program order, and in a single thread, then any consequence of action a is visible to action b
  - If action a is visible to action b, and action b is visible to action c, then action a is visible to action c (this is the transitive part)
- Notice that these rules allow flexibility in reordering instructions, and allows for normal cache operation in a single thread
- Beyond this core, some actions will exist in the model that create an "is visible to" relationship across threads
  - Such a relationship constrains reordering and might force some cache synchronization
  - Remember that these relationships are transitive – this is important!

Warning!

- The ***absence*** of a visibility relationship is not guaranteed to "***hide***" a write, but the ***presence*** of one is necessary to ***reliably*** see the write
- Consequently, if an arbitrary write operation, without a visibility relationship (i.e. a ***race***), executes in any thread prior to the read operation executing, that write ***might*** be seen by the read "by accident/luck"
- Which means that such a racy write can overwrite a properly coded write that has a visibility relationship with your read

Any data that will be shared between threads must always be treated with care and respect of the three concerns even if this particular operation is not intended to be shared

The typical visibility model - 2

Visible to relationships across threads might be provided by a combination of language constructs and library features

- If library features are provided and suitable they're often equally fast, and easier to use
- In Java and other C-derived languages, the keyword volatile typically creates a visibility relationship between a write from one thread, and a subsequent read by another thread
- Many locking structures create a visibility relationship from the release of the lock by a thread and its subsequent acquisition by another thread
- The process of starting a thread is commonly visible to the newly started thread, so data in the creator thread will be accessible to the created thread
- The last thing a thread did before it terminated is typically visible to a thread that identifies that death, so the data of the dead thread can be read

Take care: these relationships are likely only defined as between the two participating threads, but in practice might have broader consequences, potentially tricking you into thinking your code "works"

Visibility (without addressing timing)

- Java, and some other languages that share a C heritage, a modifier `volatile` can be applied to a variable and indicates that any read of that variable must pick up the most recent write by any thread
  - This modifier does not impose any timing constraints, nor protect against transactional problems

Visibility (without addressing timing)

visibility.Example1

```
private /*volatile*/ static boolean stop = false;
public static void main(String[] args) throws Throwable {
  Runnable stopper = () -> {
    System.out.println("Stopper task starting");
    while (!stop)
      ; // "spin wait" here until flag is set true
    System.out.println("Stopper task exiting");
  };
  Thread.ofPlatform().start(stopper);
  Thread.sleep(1_000);
  System.out.println("Setting stop flag");
  stop = true;
  System.out.println("Stop flag is " + stop + " main exiting");
}
```

Typical visibility models

- Both the Java Virtual Machine and the Go Programming Language are built on memory models following the broad principles just outlined although their terminology differs somewhat from that used here
- https://go.dev/ref/mem
  - If you must read the rest of this document to understand the behavior of your program, you are being too clever.
  - Don't be clever.
- https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html#jls-17.4
  - Note that Java and the JVM use the term "happens before" to describe the is visible to relationship–do not be fooled to think this means "executes prior to", that is not guaranteed
  -

Happens-before is not executes-before!

- In the JVM terminology of "happens-before" it's easy to be fooled into thinking this means "executes-before", but it ***does not***!
- If the code of thread a describes action a, and the code of thread b describes action b, and a happens before relationship exists between action a and action b, it's tempting to think that action a must execute before action b but this is not the case
  - If action b does not care about the result of action a, then execution of action a can be deferred (or possibly elided entirely)
  - If the result of action a is predictable by the compiler, then action a might not be executed at runtime at all; action b might be fed the result directly, and any other thread looking for a consequence of the execution will be disappointed
- In practice, these are corner cases, but faulty assumptions create bugs, and threading bugs are notoriously hard to find

Alternative concurrency models

- If only a single thread will exist (that is, pure concurrency, not parallelism), then the problem arises of how to keep the CPU busy doing productive work while one sub-task cannot proceed until some other operation completes
  - Threads certainly can do this, and are used for this purpose
- In this situation, an alternative approach is to package the work into separate chunks such that each chunk can run to completion without any risk of waiting for a result
- When a result is needed, the block of code issues a request to perform the work and sends another chunk of code along with the request saying "when this work completes", here's the next step
- This approach is used in the "event driven" approach used by JavaScript
- The critical part of this model is that each chunk of code must be self-contained, not block, and must not complete with any data in a transactionally unsound state
- In a single threaded system, visibility is not a problem (remember that visibility is an issue across threads)
- A similar approach is used by the "Promise Pipeline" used by JavaScript and Java's CompletableFuture (although CompletableFuture is a parallel solution, and addresses visibility across the communication points)